ScriptPoker

Introduction

The central idea behind ScriptPoker was to create a language that simplifies implementing games that use a standard poker deck. The language provides built-in handling for cards, hands, piles, and a single deck of poker cards.

ScriptPoker draws significant influence from COBOL and modern languages like C++. COBOL provides an easy-to-read style syntax, and the formatting of C++ with blocks and parenthetical function calls makes it easier to distinguish code structure.

Semantic Behavior

The language is generally case-insensitive. Variable names remain case-sensitive and must match the case used in their declaration. Card selectors written in source code are typically shown in uppercase, such as "KH" or "7S".

Statements are terminated by a newline. In practice, this means the source file must end with a newline, so the last line of a program should be blank.

Variables must always be declared before use. Variable names must begin with a letter, or an underscore followed by a letter. It may continue with letters, digits, or single underscores between alphanumeric characters. It may not end with an underscore or contain two underscores in a row.

- *varname* is valid
- *_varname* is valid
- *_var_name* is valid
- *_var__name* is invalid (double underscore)
- *varname_* is invalid (ending in underscore)

Every variable must be declared with a type before it is used. ScriptPoker uses explicit static typing, so a variable keeps the type it was declared with. Variables have block scope, and an inner declaration may shadow an outer declaration. Only one declaration is allowed per line.

Standalone blocks enclosed in braces are valid statements. They are useful for creating a smaller scope, especially when you want temporary variables that should not remain visible after the block ends.

```
define i as integer
put 3 into i

{
  define i as real
  put 5.2 into i
  display i
}
```

```
display i
```

Data Types

Basic Types
- Boolean
- Integer
- Real
- String
- Array

Complex Types
- Deck
- Hand
- Pile
- Card
- CSV
- Scoreboard

Expressions

Arithmetic
- expr + expr
- expr - expr
- expr * expr
- expr / expr
- expr DIV expr
- expr % expr

Comparison
- expr = expr
- expr <> expr
- expr < expr
- expr <= expr
- expr > expr
- expr >= expr

Logical
- expr AND expr
- expr OR expr
- NOT expr
- TRUE
- FALSE

Other
- expr & expr
- (expr)
- integer
- real
- string
- identifier

Declarations and Definitions

DEFINE: declares a variable of a specific data type.

```
// examples of some declarations
define i as integer
define str as string
define lib as deck with back "??"
define arr as array of real
define sb as scoreboard descending
```

FUNCTION: declares a function and its return type. A function call can appear anywhere an expression of the returned type is valid. Function parameters must also be declared with types, and **RETURN** supplies the function result.

PROCEDURE: declares a procedure without a return type and are executed as statements.

RETURN: returns a value from a function.

```
function add(a as integer, b as integer) returns integer {
  return a + b
}

procedure announce(a as integer) {
  display "Sum of " & a & " and " & a + 1 & " is " & add(a, a + 1)
}

define i as integer
iterate i from 1 to 5 {
  announce(i)
}
```

```
// output
// Sum of 1 and 2 is 3
// Sum of 2 and 3 is 5
// Sum of 3 and 4 is 7
// Sum of 4 and 5 is 9
// Sum of 5 and 6 is 11
```

Assignment and Data Movement

PUT: assigns a value to a variable. For most data types, **PUT** assigns a single value to a variable. (See **SCOREBOARD** for the one exception).

```
// assignment to a variable of most data type
define i as integer
put 5 into i
put (19 DIV 4) % 3 into i
```

Input and Output

DISPLAY: outputs an expression to the console and ends with newline.

PROMPT: outputs an expression to the console, then waits for the user to input text and saves the parsed value into the variable. Plain PROMPT will keep asking until valid input is entered. PROMPT with ON ERROR will run its recovery block instead of reprompting when parsing fails.

SHOW: outputs all cards in a collection as a concatenated string delimited by a space. The optional **REVEALING** keyword can limit the number of cards displayed face-up. Any face down cards will show the deck's back option for each face-down card.

```
define d as deck
define h as HAND
define i as integer

shuffle d

prompt "How many cards to deal? " into i
deal i to h

display show h
display show h revealing 1

// output
// How many cards to deal? 5
// 9♦ K♥ 6♠ J♥ 3♣
// 9♦ [] [] [] []
```

Control Flow and Blocks

IF ... ELSE: conditionally executes the **IF** block if a condition is true, or the **ELSE** block if the condition is false. The **ELSE** block is optional.

```
define cond as boolean
```

```
put true into cond

if cond {
  display "The condition was true"
} else {
  display "The condition was false"
}

// output
// The condition was true
```

EXAMINE: conditionally executes the block of the first **CASE** that evaluates to **TRUE**. If none of the cases evaluate to **TRUE**, the **OTHERWISE** block will be executed. The **OTHERWISE** block is optional.

CASE: defines the condition to match to execute a block of code in an **EXAMINE** block.

```
define i as integer
put random from 1 to 10 into i

examine {
  case i < 4 {
    display "A value of " & i & " is low"
  }
  case i > 7 {
    display "A value of " & i & " is high"
  }
  otherwise {
    display "A value of " & i & " is mid"
  }
}

// output (value will be random)
// A value of 5 is mid
```

WHILE: executes a block repeatedly while its condition evaluates to **TRUE**.

```
define i as integer
while i < 4 {
  display i
  put i + 1 into i
}

// output
// 0
// 1
// 2
// 3
```

ITERATE: executes a block a counted number of times, moving from the starting expressiong to the ending expression.

```
define i as integer
iterate i from 3 to 7 {
  display i
}
```

```
// output
// 3
// 4
// 5
// 6
// 7
```

EXIT LOOP: exits an ITERATE or WHILE loop, continuing with the next statement after the loop.

```
define i as integer
iterate i from 1 to 10 {
  display i
  if i % 4 = 0 {
    exit loop
  }
}

// output
// 0
// 1
// 2
// 3
```

File Input and Output

ScriptPoker currently supports one file type: comma-separated values (csv). The file I/O type is **CSV**. By default, the field delimiter is a comma, but a single-character delimiter may be provided with the **DELIMITER** option when the **CSV** is defined.

Fields are read from the **CSV** one line at a time and assigned to the listed variables in order. Extra fields in the file are ignored. If fewer fields are present than variables requested, the remaining variables are set to their default values. **READ** and **WRITE** currently support **INTEGER**, **REAL**, **BOOLEAN**, and **STRING** fields. If a field cannot be parsed as the requested type, an error is raised.

Data is written with **WRITE** in both **WRITE** and **APPEND** modes. Fields are separated with the configured delimiter, or with a comma when no delimiter was specified.

CSV: data type used for files that may be opened for **READ**, **WRITE**, or **APPEND** operations. A file must be closed before it is reopened in a different mode.

OPEN: opens a **CSV** file for **READ**, **WRITE**, or **APPEND** operations. The mode is specified in the **OPEN** statement. A file must be closed before it can be open with another mode.

CLOSE: closes an open file.

READ: reads a line of text from a **CSV** file and assigns them to the variable list in order. If there are more fields in the **CSV** than variables in the variable list, the extra fields are discarded. If there are more variables in the variable list than there are fields read on a line, the extra variables are set to their default values.

WRITE: writes a variable list to a line in the **CSV** file.

END OF: returns a **BOOLEAN** to indicate if the end of a **CSV** file has been reached.

```
define file_hndl as csv delimiter("|")

put "myfile.txt" into file_hndl

open file_hndl for write
write "field1", "field2", "field3" to file_hndl
close file_hndl

define f1 as string
define f2 as string
define f3 as string

open file_hndl for read
while not end of file_hndl {
  read f1, f2, f3 from file_hndl
  display f1 & " " & f2 & " " & f3
}
close file_hndl

// output
// field1 field2 field3
```

Arrays

ScriptPoker uses dynamic, 1-based arrays. Each **ARRAY** has a single declared element type, and arrays cannot be declared with a fixed size. Nested arrays (an array of arrays) and multidimensional arrays are not supported.

Elements can be added to an array by assigning a value to the next index after the current end of the array. Assigning to an index beyond that next position raises an error.

Elements of an array can be set with an array literal.

```
define arr as array of integer

put [4, 5, 3, 7] into arr
put 5 into arr[count of arr + 1]

display arr
display arr[3]

// output
// [4, 5, 3, 7, 5]
// 3
```

Utility

RANDOMIZE: sets the seed for random umber. If a seed value is not provided, the system timer is used.

RANDOM: generates a random number. RANDOM by itself returns a REAL value greater than or equal to 0 and less than 1. Providing an integer range with FROM and TO returns an INTEGER chosen inclusively within that range.

```
define i as integer
define r as real
```

```
randomize with 42

put random from 1 to 10 into i
put random into r


display "random i: " & i
display "random r: " & r

// output
// random i: 9
// random r: 0.7275636800328681
```

ON ERROR: Attaches a recovery block to certain runtime-failing statements. Statements that allow a recovery block are PROMPT, OPEN, READ, WRITE, CLOSE, DEAL, DRAW, MOVE, TAKE, and BURY. Helps to catch runtime errors that could halt the program.

```
define file as csv
put "nofile.txt" into file

open file for read on error {
    display "error attempting to open file"
}

display "end of line"

// output runtime error without the on error block
// Error: CSV open failed for nofile.txt: nofile.txt

// output with on error block
// error attempting to open file
// end of line
```

COUNT OF: can be used to get the number of cards in a **HAND**, **PILE**, or **DECK**, the number of elements in an **ARRAY**, or the number of entries in a **SCOREBOARD**.

```
define d as deck
define arr as array of integer

put [5, 2, 7, 9] into arr
move 12 random from d to discard

display "size of arr: " & count of arr
display "size of discard: " & count of discard

// output
// size of arr: 4
// size of discard: 12
```

Deck Definitions

DECK: this data type is the core domain object in ScriptPoker. Only one **DECK** may be defined in a program, and it must be defined before any **HAND**, **PILE**, or **CARD** variable can be declared. The **DECK** definition itself is

fixed after creation, but the deck's card contents change during execution as cards are dealt, moved, buried, shuffled, and recovered.

DECK can be defined without options. This creates the standard 52-card **DECK** without jokers. By default, **CARD** values are A=1, 2=2, ... 10=10, J=11, Q=12, and K=13.

```
define d as deck
define i as integer

put 1 into i

while i <= count of d {
    display card i of d & " has a value of " & value of card i of d
    put i + 1 into i
}

// output
// A♠ has a value of 1
// 2S has a value of 2
// 3S has a value of 3
// ...
// JH has a value of 11
// QH has a value of 12
// KH has a value of 13
```

WITH: sets the options for a **DECK** when it's declared. Options may be provided in any order, but the runtime applies them in the following order when building a **DECK**.

- **JOKERS**: adds the two joker cards, XB and XL, to the base **DECK** before the other card-selection options are applied.
- **REMOVE**: removes cards using an array of selectors. A **SUIT** selector removes an entire **SUIT**, a **RANK** selector removes an entire **RANK**, and an exact **CARD** selector removes one specific **CARD** from the base shoe.
- **VALUES**: Assigns one value to cards selected by **SUIT**, **RANK**, or exact **CARD**. Exact-card selectors override **RANK** selectors, and **RANK** selectors override **SUIT** selectors. Repeating the same selector later in the **VALUES** list replaces the earlier value for that selector. **VALUES** can assign a two-value rule to cards selected by **SUIT**, **RANK**, or exact **CARD**. The first number is the default **VALUE OF** result. **VALUE HIGH OF** returns the higher of the two numbers, and **VALUE LOW OF** returns the lower of the two numbers. These alternate values are especially useful with **TOTAL ... THRESHOLD** calculations.
- **SHOES**: sets how many copies of the configured base shoe are included. For example, **SHOES** 2 creates a 104-card **DECK** when no jokers are used.
- **BACK**: sets the string used when a face-down card is shown. By default, the **BACK** display is "[]".

```
// this example would not compile because only one deck
// can be declared in a program. Each line by itself
// would compile and execute.

define d as deck
define d as deck with shoes 2
define d as deck with jokers back "??"
define d as deck with values ["K":5, "D":10, "KD":0]
define d as deck with remove ["A", ""K", "Q", "J"]
```

```
define d as deck with shoes 2 jokers values ["H":20] remove ["10"] back "="
```

Containers and Card Access

The term **CONTAINER** is not used in the language, but is used in documentation to reference an expression that could be a **HAND**, **PILE**, or **DECK**. Referencing an index out of range of a **CONTAINER** will raise an error.

CARD: references a specific card by index for a **CONTAINER**. If index is not provided, index of 1 is assumed.

Count Of: returns the number of cards in a container. SIZE is an alias of COUNT.

```
define d as deck
define h as hand

deal 4 to h
display show h
display count of h

// output
// AS 2S 3S 4S
// 4
```

SHOW: concatenates the cards of a **CONTAINER** into a space-delimited **STRING**. Optionally, the number of cards shown face-up can be set using **REVEALING**.

```
define d as deck

move 10 random from d to discard

display "discard pile: " & show discard revealing 3

// output
// discard pile: 3♠ 9♥ 10♦ [] [] [] [] [] [] []
```

RANK OF CARD: displays the **RANK** of the **CARD** at the specified index of a **CONTAINER**. If the index is not provided, index of 1 is assumed.

SUIT OF CARD: displays the **SUIT** of the **CARD** at the specified index of a **CONTAINER**. If the index is not provided, index of 1 is assumed.

```
define d as deck
define h as hand

deal to h
display show h
display rank of card of h
display suit of card of h

// output
// AS
// A
// S
```

VALUE OF CARD: displays the default **VALUE** of the **CARD** at the specified index of a **CONTAINER**. If the index is not provided, index of 1 is assumed.

VALUE HIGH OF CARD: displays the high **VALUE** of the **CARD** at the specified index of a **CONTAINER**. If the index is not provided, index of 1 is assumed.

VALUE LOW OF CARD: displays the low **VALUE** of the **CARD** at the specified index of a **CONTAINER**. If the index is not provided, index of 1 is assumed.

```
// aces are define with a low value of 1 and a high value of 11
// the default value is 1 since it appears first in the assignment

define d as deck with values ["A":[1:11]]

display "First card of deck is " & card of d
display "It's default value is " & value of card of d
display "It's high value is " & value high of card of d
display "It's low value is " & value low of card of d

// output
// First card of deck is A♠
// It's default value is 1
// It's high value is 11
// It's low value is 1
```

Movement and Placement

Several statements support card selections. Multiple selectors can be entered as a selector list if delimited by a comma.

- A specific **CARD** name like "AC" or "5H"
- An entire **RANK** of cards. "5" will select all 5's regardless of **SUIT**.
- An entire **SUIT** of cards, "H" will select all hearts regardless of **RANK**.

Pattern checks use CHECK … FOR PATTERN(…) to test for **CARD** structures in a HAND or PILE.

CHECK: returns TRUE when the container satisfies the listed pattern terms and FALSE otherwise. Pattern terms may be combined, and the runtime tries to satisfy them without reusing the same card twice unless a joker is filling a gap.

> **RANK(N <, SELECTOR>)**: checks for n cards of the same **RANK**. The optional selector limits the **RANK**, such as "A".
> **SUIT(N <, SELECTOR>)**: checks for n cards of the same **SUIT**. The optional selector limits the **SUIT**, such as "H".
> **RUN(N <, SELECTOR>)**: checks for a run of n consecutive ranks. The optional selector requires the run to include the named **RANK**.
> **CARD(N, SELECTOR)**: checks for n copies of one exact **CARD** selector, such as "AS".

Invalid runtime arguments for pattern terms do not raise an error. Instead, the pattern check safely evaluates to **FALSE**. Jokers behave as wild cards in pattern matching.

```
define h as hand
```

```
move "AS", "AD", "4C", "5H", "6S" from d to h
display show h
display check h for pattern(rank(2), run(3))

// output
// A♠ A♦ 4♣ 5♥ 6♠
// TRUE
```

DEAL: moves the number of cards from the top of the **DECK** to the **CONTAINER**. If the number of cards is not specified, 1 is assumed. **DRAW** is an alias of **DEAL**.

MOVE: moves specified cards from one **CONTAINER** to another. **MOVE** supports multiple ways of selecting cards. **TAKE** is an alias of **MOVE**.

- **MOVE ALL** moves all cards from the **CONTAINER**.
- **MOVE ALL** selector_list: moves all cards that match the card selectors from the **CONTAINER**.
- **MOVE** selector_list: moves one matching card for each selector in selector_list from the **CONTAINER**.
- **MOVE** x **RANDOM**: selects x random cards from the **CONTAINER** to move.
- **MOVE** x **RANDOM TOP** y: selects x random cards from the top y cards of the **CONTAINER**.

```
define d as deck
define h as hand

shuffle d

deal 20 to h
display show h

// move specific cards
move "JC", "5S" from h to discard

// move all hearts and all 2s
move ALL "H", "2" from h to discard

// move 2 randoms cards from the top 5 cards
move 2 random top 5 from h to discard

display show discard

// output
// 7♥ 2♦ A♣ J♣ 6♦ 5♠ J♦ 2♠ 5♥ 8♥ 2♣ K♦ 10♦ 5♣ 4♦ 7♦ 5♦ Q♦ 8♦ 10♣
// J♣ 5♠ 7♥ 5♥ 8♥ 2♦ 2♠ 2♣ K♦ A♣
```

BURY: moves cards from a source **CONTAINER** into a **DECK** or **PILE**. **BURY** supports the same selection forms as **MOVE**, and it also supports two independent **RANDOMLY** positions.

- **BURY ALL**: moves all cards in the source **CONTAINER** to the target **CONTAINER**. When the target is a **DECK**, plain **BURY** places them on the bottom in their current order.
- **BURY ALL** selector_list: moves all cards that match the selector list to the target **CONTAINER**. Without **RANDOMLY**, they keep their current order.
- **BURY** x **RANDOM**: moves x random cards from the source **CONTAINER** to the target **CONTAINER**.
- **BURY** x **RANDOM TOP** y: moves x random cards chosen from the **TOP** y cards of the source **CONTAINER** to the target **CONTAINER**.

- **BURY** selection **RANDOMLY FROM** source **TO** target: shuffles the moved-**CARD** group before it is placed into the target.
- **BURY** selection **FROM** source **RANDOMLY TO** target: inserts the moved cards at random positions in the target **CONTAINER**.

```
define d as deck
define h as hand

deal 52 to h

bury all "A" from h to d
display show d

bury 2 random top 5 from h to d
display show d

bury 3 random from h randomly to d
display show d

// output
// A♠ A♦ A♣ A♥
// A♠ A♦ A♣ A♥ 3♠ 5♠
// 4♦ A♠ A♦ J♣ 9♦ A♣ A♥ 3♠ 5♠
```

Scoreboard

SCOREBOARD: stores ordered entries made of a player name and an integer score. By default, higher scores sort first. **ASCENDING** may be specified on declaration to reverse that ordering. New entries are inserted into their sorted position as they are added.

PUT <STRING>, <INTEGER> INTO <SCOREBOARD>: adds a **STRING**-**INTEGER** pair to the **SCOREBOARD**.

PUT ENTRY <INDEX> OF <SCOREBOARD> INTO <STRING>, <INTEGER>: retrieves a **STRING**-**INTEGER** pair from the **SCOREBOARD** at the specified index.

```
define i as integer
define name as string
define score as integer
define highscores as scoreboard

put "Bob", 250 into highscores
put "Neal", 150 into highscores
put "Jim", 300 into highscores

put 1 into i
while i <= count of highscores {
    put entry i of highscores into name, score
    display name & ": " & score
    put i + 1 into i
}

// output
// Jim: 300
// Bob: 250
```

```
// Neal: 150
```

BNF Information

Keywords in source code are case-insensitive. This BNF uses canonical uppercase spellings for reserved words.

```
<program> ::= (<blank_line> | <statement> <newline>)*

<blank_line> ::= <newline>

<newline> ::= "\n" | "\r\n"

<block> ::= "{" <newline>* (<blank_line> | <statement> <newline>)* "}"

<statement> ::= <define_statement>
Â  Â  Â  Â  | <put_statement>
Â  Â  Â  Â  | <open_statement>
Â  Â  Â  Â  | <read_statement>
Â  Â  Â  Â  | <write_statement>
Â  Â  Â  Â  | <close_statement>
Â  Â  Â  Â  | <display_statement>
Â  Â  Â  Â  | <prompt_statement>
Â  Â  Â  Â  | <if_statement>
Â  Â  Â  Â  | <examine_statement>
Â  Â  Â  Â  | <while_statement>
Â  Â  Â  Â  | <iterate_statement>
Â  Â  Â  Â  | <exit_loop_statement>
Â  Â  Â  Â  | <shuffle_statement>
Â  Â  Â  Â  | <recover_statement>
Â  Â  Â  Â  | <deal_statement>
Â  Â  Â  Â  | <move_statement>
Â  Â  Â  Â  | <bury_statement>
Â  Â  Â  Â  | <randomize_statement>
Â  Â  Â  Â  | <function_decl>
Â  Â  Â  Â  | <procedure_decl>
Â  Â  Â  Â  | <return_statement>
Â  Â  Â  Â  | <procedure_call>
Â  Â  Â  Â  | <block>

<define_statement> ::= "DEFINE" <identifier> "AS" <type_name>
Â  Â  Â  Â  | "DEFINE" <identifier> "AS" "CSV" ("DELIMITER" <expr>)?
Â  Â  Â  Â  | "DEFINE" <identifier> "AS" "SCOREBOARD" ("ASCENDING" |
"DESCENDING")?
Â  Â  Â  Â  | "DEFINE" <identifier> "AS" "DECK" ("WITH" <deck_list_options>)?

<put_statement> ::= "PUT" <expr> "INTO" <assign_target>
Â  Â  Â  Â  | "PUT" <expr> "," <expr> "INTO" <identifier>
Â  Â  Â  Â  | "PUT" "ENTRY" <expr> "OF" <identifier> "INTO" <identifier> ","
<identifier>

<display_statement> ::= "DISPLAY" <expr>

<prompt_statement> ::= "PROMPT" <prompt_message>? "INTO" <identifier>
```

```
Â  Â  Â  Â  | "PROMPT" <prompt_message>? "INTO" <identifier> "ON ERROR"
<block>

<open_statement> ::= "OPEN" <identifier> "FOR" ("READ" | "WRITE" | "APPEND")
Â  Â  Â  Â  | "OPEN" <identifier> "FOR" ("READ" | "WRITE" | "APPEND") "ON
ERROR" <block>

<read_statement> ::= "READ" <identifier_list> "FROM" <identifier>
Â  Â  Â  Â  | "READ" <identifier_list> "FROM" <identifier> "ON ERROR" <block>

<write_statement> ::= "WRITE" <expr_list> "TO" <identifier>
Â  Â  Â  Â  | "WRITE" <expr_list> "TO" <identifier> "ON ERROR" <block>

<close_statement> ::= "CLOSE" <identifier>
Â  Â  Â  Â  | "CLOSE" <identifier> "ON ERROR" <block>

<if_statement> ::= "IF" <expr> <block> ( "ELSE" <block> )?

<examine_statement> ::= "EXAMINE" "{" <case_clause>+ <otherwise_clause>? "}"

<case_clause> ::= "CASE" <expr> <block>

<otherwise_clause> ::= "OTHERWISE" <block>

<while_statement> ::= "WHILE" <expr> <block>

<iterate_statement> ::= "ITERATE" <identifier> "FROM" <expr> "TO" <expr>
<block>

<exit_loop_statement> ::= "EXIT LOOP"

<shuffle_statement> ::= "SHUFFLE" <identifier>

<recover_statement> ::= "RECOVER" <identifier>

<deal_statement> ::= ("DEAL" | "DRAW") <expr>? "TO" (<container_target> |
"ALL HANDS")
Â  Â  Â  Â  | ("DEAL" | "DRAW") <expr>? "TO" (<container_target> | "ALL
HANDS") "ON ERROR" <block>

<move_statement> ::= ("MOVE" | "TAKE") <move_selection> "FROM"
<container_target> "TO" <container_target>
Â  Â  Â  Â  | ("MOVE" | "TAKE") <move_selection> "FROM" <container_target>
"TO" <container_target> "ON ERROR" <block>

<bury_statement> ::= "BURY" <move_selection> "RANDOMLY"? "FROM"
<container_target> "RANDOMLY"? "TO" <container_target>
Â  Â  Â  Â  | "BURY" <move_selection> "RANDOMLY"? "FROM" <container_target>
"RANDOMLY"? "TO" <container_target> "ON ERROR" <block>

<randomize_statement> ::= "RANDOMIZE" ("WITH" <expr>)?

<function_decl> ::= "FUNCTION" <identifier> "(" <parameter_list>? ")"
"RETURNS" <type_name> <block>

<procedure_decl> ::= "PROCEDURE" <identifier> "(" <parameter_list>? ")"
<block>
```

```
<return_statement> ::= "RETURN" <expr>

<procedure_call> ::= <call_expr>

<prompt_message> ::= <expr>

<deck_list_options> ::= <deck_option> <deck_option>*

<deck_option> ::= "SHOES" <expr>
Â  Â  Â  Â  | "JOKERS"
Â  Â  Â  Â  | "VALUES" <value_map>
Â  Â  Â  Â  | "REMOVE" <expr>
Â  Â  Â  Â  | "BACK" <expr>

<value_map> ::= "[" <value_entry_list> "]"

<value_entry_list> ::= <value_entry> ("," <value_entry>)*

<value_entry> ::= <string_literal> ":" <value_spec>

<value_spec> ::= <expr>
Â  Â  Â  Â  | "[" <expr> ":" <expr> "]"

<move_selection> ::= "ALL"
Â  Â  Â  Â  | "ALL"? <expr_list>
Â  Â  Â  Â  | <expr>? "RANDOM" <move_limit>?

<move_limit> ::= "TOP" <expr>

<pattern_list> ::= <pattern_term> ("," <pattern_term>)*

<pattern_term> ::= "RANK" "(" <expr> ("," <expr>)? ")"
Â  Â  Â  Â  | "SUIT" "(" <expr> ("," <expr>)? ")"
Â  Â  Â  Â  | "RUN" "(" <expr> ("," <expr>)? ")"
Â  Â  Â  Â  | "CARD" "(" <expr> "," <expr> ")"

<parameter_list> ::= <parameter> ("," <parameter>)*

<parameter> ::= <identifier> "AS" <type_name>

<type_name> ::= "INTEGER"
Â  Â  Â  Â  | "REAL"
Â  Â  Â  Â  | "STRING"
Â  Â  Â  Â  | "BOOLEAN"
Â  Â  Â  Â  | "CSV"
Â  Â  Â  Â  | "ARRAY" "OF" <type_name>
Â  Â  Â  Â  | "CARD"
Â  Â  Â  Â  | "HAND"
Â  Â  Â  Â  | "PILE"
Â  Â  Â  Â  | "DECK"
Â  Â  Â  Â  | "SCOREBOARD"

<assign_target> ::= <identifier>
Â  Â  Â  Â  | <index_expr>

<container_target> ::= <identifier>
```

```
Â  Â  Â  Â  | "DISCARD"

<expr> ::= <or_expr>

<or_expr> ::= <or_expr> "OR" <and_expr>
Â  Â  Â  Â  | <and_expr>

<and_expr> ::= <and_expr> "AND" <compare_expr>
Â  Â  Â  Â  | <compare_expr>

<compare_expr> ::= <concat_expr> <compare_op> <concat_expr>
Â  Â  Â  Â  | "NOT" <compare_expr>
Â  Â  Â  Â  | <concat_expr>

<compare_op> ::= "="
Â  Â  Â  Â  | "<>"
Â  Â  Â  Â  | "<"
Â  Â  Â  Â  | "<="
Â  Â  Â  Â  | ">"
Â  Â  Â  Â  | ">="

<concat_expr> ::= <concat_expr> "&" <addsub_expr>
Â  Â  Â  Â  | <addsub_expr>

<addsub_expr> ::= <addsub_expr> "+" <multdiv_expr>
Â  Â  Â  Â  | <addsub_expr> "-" <multdiv_expr>
Â  Â  Â  Â  | <multdiv_expr>

<multdiv_expr> ::= <multdiv_expr> "*" <unary_expr>
Â  Â  Â  Â  | <multdiv_expr> "/" <unary_expr>
Â  Â  Â  Â  | <multdiv_expr> "DIV" <unary_expr>
Â  Â  Â  Â  | <multdiv_expr> "%" <unary_expr>
Â  Â  Â  Â  | <unary_expr>

<unary_expr> ::= "-" <unary_expr>
Â  Â  Â  Â  | <primary_expr>

<primary_expr> ::= <integer_literal>
Â  Â  Â  Â  | <real_literal>
Â  Â  Â  Â  | <boolean_literal>
Â  Â  Â  Â  | <string_literal>
Â  Â  Â  Â  | <array_literal>
Â  Â  Â  Â  | <index_expr>
Â  Â  Â  Â  | <identifier>
Â  Â  Â  Â  | <card_selector>
Â  Â  Â  Â  | <check_expr>
Â  Â  Â  Â  | "SHOW" <container_target> ("REVEALING" <addsub_expr>)?
Â  Â  Â  Â  | <property_expr>
Â  Â  Â  Â  | <random_expr>
Â  Â  Â  Â  | <call_expr>
Â  Â  Â  Â  | "(" <expr> ")"

<check_expr>   ::=   "CHECK"   <container_target>   "FOR"   "PATTERN"   "("
<pattern_list>? ")"

<property_expr> ::= ("COUNT" | "SIZE") "OF" <property_target>
Â  Â  Â  Â  | "END" "OF" <identifier>
```

```
Â  Â  Â  Â  | "RANK" "OF" <property_target>
Â  Â  Â  Â  | "SUIT" "OF" <property_target>
Â  Â  Â  Â  | "VALUE" "HIGH" "OF" <property_target>
Â  Â  Â  Â  | "VALUE" "LOW" "OF" <property_target>
Â  Â  Â  Â  | "VALUE" "OF" <property_target>
Â  Â  Â  Â  | "TOTAL" <value_mode>? "OF" <property_target> ("REVEALING"
<addsub_expr>)? ("THRESHOLD" <expr>)?
Â  Â  Â  Â  | "ACES" <value_mode> "OF" <property_target> "THRESHOLD" <expr>

<card_selector> ::= "CARD" <expr>? "OF" <container_target>

<property_target> ::= "(" <expr> ")"
Â  Â  Â  Â  | <card_selector>
Â  Â  Â  Â  | <container_target>
Â  Â  Â  Â  | <call_expr>
Â  Â  Â  Â  | <index_expr>
Â  Â  Â  Â  | <identifier>

<value_mode> ::= "HIGH" | "LOW"

<random_expr> ::= "RANDOM" ("FROM" <expr> "TO" <expr>)?

<index_expr> ::= <identifier> "[" <expr> "]"

<call_expr> ::= <identifier> "(" <argument_list>? ")"

<argument_list> ::= <expr> ("," <expr>)*

<identifier_list> ::= <identifier> ("," <identifier>)*

<string_literal> ::= "\"" <string_char>* "\""

<string_char> ::= any printable character except "\"" and newline

<array_literal> ::= "[" <expr_list>? "]"

<expr_list> ::= <expr> ("," <expr>)*

<boolean_literal> ::= "TRUE" | "FALSE"

<integer_literal> ::= <digit>+

<real_literal> ::= <integer_literal> "." <integer_literal>

<identifier> ::= "_"? <letter> ("_"? <alphanumeric>)*

<letter> ::= "a" | ... | "z" | "A" | ... | "Z"

<digit> ::= "0" | ... | "9"

<alphanumeric> ::= <letter> | <digit>
```